

Fundamentos de Organización de Datos

- [Conceptos generales de archivos](#)
 - [Características](#)
 - [¿Cómo se manipula un archivo desde un programa?](#)
 - [Acceso a la información](#)
 - [Tipos de archivos](#)
 - [Buffers](#)
- [Algoritmia clásica](#)
 - [Actualización \(maestro y detalle\)](#)
 - [Merge](#)
 - [Corte de control](#)
- [Eliminación de datos \(registros de longitud fija\)](#)
 - [Bajas físicas](#)
 - [Bajas lógicas](#)
 - [Recuperación y reasignación de espacio](#)
- [Registros y campos de longitud variable](#)
 - [Funcionamiento](#)
 - [Bajas](#)
 - [Modificaciones](#)
- [Búsqueda de información](#)
 - [Búsqueda en distintos tipos de archivos](#)
- [Índices](#)
 - [Índices primarios](#)
 - [Operaciones](#)
 - [Índice para claves candidatas](#)
 - [Índices secundarios](#)
 - [Operaciones](#)
 - [Alternativas](#)
 - [Índices selectivos](#)
- [Árboles binarios](#)
 - [Problemas de árboles binarios](#)
 - [Árboles AVL](#)
 - [Paginación de árboles binarios](#)
 - [Problema de los árboles paginados](#)
- [Árboles multicamino](#)
 - [Árboles B](#)
 - [Implementación en Pascal](#)
 - [Eficiencia de búsqueda](#)

- [Eficiencia de inserción](#)
 - [Eficiencia de eliminación](#)
 - [Eficiencia de modificación](#)
- [Árboles B*](#)
- [Árboles B+](#)
 - [Árboles B+ de prefijos simples](#)
- [Relación entre árboles y buffers](#)
- [Dispersión \(*hashing*\)](#)
 - [Propiedades](#)
 - [Limitaciones](#)
 - [Tipos de dispersión](#)
 - [Parámetros de la dispersión](#)
 - [Función de *hash*](#)
 - [Tamaño de cada nodo/celda/*bucket*](#)
 - [Densidad de empaquetamiento](#)
 - [Método de tratamiento de *overflow*](#)
 - [Hashing con espacio de direccionamiento dinámico](#)
- [Conclusiones](#)

Conceptos generales de archivos

Características

- Es una estructura de datos homogénea.
- Se guarda en el almacenamiento secundario (*persistente*) de la computadora.
Ej: disco rígido/sólido.
 - Esto implica una gran capacidad de almacenamiento, pero una velocidad de acceso órdenes de magnitud más lenta que la memoria RAM.
- Sobre él se pueden realizar altas, bajas, modificaciones y consultas.

¿Cómo se manipula un archivo desde un programa?

Como el archivo está almacenado en memoria secundaria, lo administra el sistema operativo. Para poder manipularlo desde un programa, debemos crear un vínculo que nos permita tratar al archivo “como si” estuviese en la memoria RAM, y así acceder a él a través de una variable. Es así que obtenemos los 2 niveles de los archivos:

- **Nivel físico:** el archivo físico, que está almacenado en la memoria secundaria, y que administra el sistema operativo.

- **Nivel lógico:** al crear un programa, se genera un vínculo entre una variable y el nombre físico del archivo con una instrucción especial (*Assign*). De esta manera, cualquier acción realizada por el algoritmo que afecte al archivo es en realidad una instrucción al sistema operativo, quien lleva a cabo la acción.
 - La acción de generar un vínculo entre una variable y el nombre físico del archivo se denomina *independencia física*.

Acceso a la información

- **Acceso secuencial (o secuencial físico):** el acceso a cada elemento de datos se realiza luego de haber accedido a su inmediato anterior. Esto se hace según el orden físico del archivo.
 - Deriva en un algoritmo de **orden lineal**
- **Acceso secuencial indizado:** el acceso a cada elemento se hace según el orden que determina un índice. Es decir, el orden físico de los datos del archivo no es relevante. Lo que importa es el orden del índice, que generalmente se almacena en otro archivo.
 - Posibilita una búsqueda de **orden logarítmico**.
- **Acceso directo:** se requiere un solo acceso para recuperar un determinado dato, sin importar el orden. Esto se hace mediante la técnica de dispersión/*hashing*.
 - Permite búsquedas de **orden constante**.

Tipos de archivos

- **Archivo serie:** archivo en el que sus elementos no tienen ningún orden más que el físico.
- **Archivo secuencial:** archivo cuyos elementos son accesibles en orden de alguna clave (acceso secuencial lógico o acceso secuencial indizado).
- **Archivo directo:** archivo en el que cualquiera de sus registros es accesible sin acceder a sus anteriores.

Buffers

- ¿Qué es un buffer?
 - Un buffer es una memoria intermedia (ubicada en RAM) entre un archivo y un programa. En él, los datos se guardan temporalmente hasta ser almacenados definitivamente en memoria secundaria. También se almacenan los datos que se recuperan de memoria secundaria.
- ¿A qué acceden las operaciones *read* y *write*?

- Cuando se ejecuta una instrucción *read* en un programa, siempre lee del buffer, ubicado en RAM. Si encuentra lo que busca, lo lee sin problema. Si no lo encuentra, el sistema operativo recupera de memoria secundaria un bloque de registros, y lo almacena en el *buffer* (esta operación se llama *input*). De esta manera, el *read* se puede ejecutar.
 - De este modo, cuando la operación *read* encuentra lo que busca en el buffer, no realiza un acceso a disco, y la operación se realiza en órdenes de nanosegundos. En cambio, si se requiere la realización de una operación *input*, se realizará en órdenes de milisegundos.
 - De esta forma, los *inputs*, que son las operaciones más costosas, se realizan con menor frecuencia.
 - El *write* funciona de la misma manera. Escribe en el buffer hasta que esté lleno. Una vez que se llena el buffer, se descarga su contenido a memoria secundaria, dejando el buffer vacío. Esta operación se llama *output*, y se puede realizar a la fuerza (aunque el buffer no esté lleno) si se ejecuta una instrucción *close*.
-

Algoritmia clásica

- Para ejecutar estos algoritmos de manera óptima, todos los archivos deben estar ordenados por un mismo criterio.
- Técnicamente, el *merge* y la actualización se pueden ejecutar con archivos desordenados.

Actualización (maestro y detalle)

- **Archivo maestro:** contiene información sobre un dominio de problema específico
- **Archivo detalle:** contiene novedades o movimientos realizados sobre la información almacenada en el maestro.
- Los algoritmos para la actualización varían según las pre-condiciones del problema y la cantidad de archivos detalles. Generalmente, si se busca únicamente la actualización, se deben recorrer los archivos detalles en su totalidad, mientras que algunos registros del archivo maestro pueden no ser procesados.
 - Usar los detalles como condición de fin es posible si se está usando un dispositivo de almacenamiento secundario con la posibilidad de

realizar accesos directos (*seek*), como lo son los discos magnéticos. En otros dispositivos, como las viejas cintas magnéticas, no se puede sobrecribir la información, así que se tendría que siempre recorrer todo el archivo maestro para copiarlo a uno nuevo (con las respectivas modificaciones del archivo detalle).

- Cuando resulta necesario leer del archivo detalle antes de comenzar la iteración, y durante la misma, surge el problema de controlar el EOF. La solución es crear un procedimiento “leer” que devuelve un valor imposiblemente alto si se llegó al fin del archivo.
- Cuando se tienen N detalles, resulta necesario definir un criterio para que todas las apariciones de una clave A se lean de manera consecutiva. La solución es siempre leer el mínimo elemento de los N detalles. Cuando ese mínimo cambia, es porque cambió la clave, y ya no hay más apariciones de la clave A. Esto es posible gracias al orden de los archivos.

Merge

- Se usa en casos en los que no existe un archivo maestro. En cambio, existen 1 o más archivos, que deben ser unidos en un nuevo archivo maestro. Estos archivos ya existentes pueden contener distintos tipos de registros, y el archivo maestro debe ser creado.
- Utiliza un procedimiento “mínimo” por los mismos motivos que el algoritmo de actualización.

Corte de control

- Se usa para presentar la información de un archivo de manera organizada de acuerdo con la estructura que tiene el mismo.

Eliminación de datos (registros de longitud fija)

- **Baja:** proceso que permite quitar información de un archivo.
- ¿Por qué son poco comunes las bajas?
 - La información es el bien máspreciado para las organizaciones que utilizan bases de datos. Es por esto que, generalmente, la información no se quita, sino que se preserva en archivos históricos.

Bajas físicas

- En este tipo de bajas, la información se borra efectivamente, recuperando el espacio de almacenamiento.
- Existen 2 maneras de realizar bajas físicas:
 - Generar un nuevo archivo que sea una copia del original, pero sin los elementos que se desean eliminar.
 - Luego de recorrer todo el archivo original, este se debe borrar de la memoria secundaria.
 - El nuevo archivo generado recibe el nombre que tenía el archivo original.
 - Desventajas
 - El disco rígido debe tener la capacidad de almacenar a los 2 archivos a la vez (el original y la copia), ya que el original se borra cuando el nuevo ya fue creado
 - *Performance*: este algoritmo es de orden lineal, porque realiza n lecturas y $n - 1$ escrituras. Ese -1 se debe a que no se escribe el elemento borrado en la nueva copia.
 - Utilizar el mismo archivo, generando los reacomodamientos necesarios.
 - Una vez que se encuentra el elemento a borrar, todos los elementos posteriores se desplazan a la posición inmediatamente anterior. Luego, con el puntero en el último elemento que se desplazó, se realiza un *Truncate* sobre el archivo. Este comando hace que se ponga la marca de EOF en la posición a la que está indicando el puntero.
 - A diferencia de la manera anterior, el disco rígido no necesita la capacidad de almacenar a los 2 archivos a la vez. Sin embargo, tiene los mismos problemas en cuanto a su *performance*. Realiza n lecturas y, en el peor caso, $n - 1$ escrituras.

Bajas lógicas

- En este tipo de bajas, la información se borra, pero sin recuperar espacio de almacenamiento.
- Funcionamiento
 - En el elemento que se desea borrar, se coloca una marca (por ejemplo, ‘***’) que indica que no está disponible. Esto se escribe en un campo del registro determinado de antemano.
- Ventaja

- La *performance* mejora respecto a las bajas físicas. Las lecturas necesarias siguen siendo n en el peor de los casos (cuando el elemento a borrar es el último). Sin embargo, la cantidad de escrituras es 1, ya que solo se escribe la marca en el elemento borrado.
- Desventaja
 - No se recupera espacio de almacenamiento, por lo que el tamaño del archivo tiende siempre a crecer.

Recuperación y reasignación de espacio

- Luego de realizar bajas lógicas, se obtiene un archivo con registros lógicamente borrados, pero que siguen ocupando espacio de almacenamiento. Existen 2 maneras de reutilizar dicho espacio.
- Recuperación de espacio:
 - Consiste en realizar periódicamente bajas físicas de todos aquellos registros borrados lógicamente, haciendo una *compactación* del archivo. Para eso se puede usar cualquiera de los métodos explicados de bajas físicas (generando una nueva copia, o usando el *Truncate*).
- Reasignación de espacio:
 - Consiste en modificar el proceso de altas, de tal manera que, si existe espacio ocupado por registros borrados, se inserte el nuevo registro allí.
 - **Desventaja:** se pierde el orden del archivo.
 - Funcionamiento
 - Se genera una lista encadenada invertida con las posiciones borradas. El registro 0 del archivo, llamado *cabecera*, contiene la información de la próxima posición a reasignar. Luego, cada posición tiene el NRR de su siguiente, mientras que el último indica el fin de la lista.
 - El proceso de alta debe tener en cuenta esto, y actualizar el registro cabecera como corresponda.

Registros y campos de longitud variable

Funcionamiento

- Surgen para minimizar el desperdicio de espacio. Si un campo de un registro como “descripción” puede llegar a ocupar 255 bytes, pero en promedio ocupa 30, en la gran mayoría de los casos se estaría desperdiciando

espacio. Como resultado, se utilizan campos de longitud variable, es decir que los campos no tienen una longitud predeterminada, sino que se ajustará a su contenido.

- Para simplificar, estos archivos con registros de longitud variable se consideran una secuencia de bytes, con EOF marcando el fin de la misma.
- Problemas
 - Los elementos de los datos se descomponen en cada uno de sus elementos consecutivos. Un string se transfiere carácter por carácter, y un número cifra por cifra. Esto dificulta el desarrollo de algoritmos para tratar con este tipo de archivos.
 - De alguna manera, se debe definir dónde empieza y dónde termina cada registro y cada campo. Generalmente se usan las *marcas de fin de campo/registro* (también llamadas delimitadores de campo/registro), que son caracteres especiales que no pueden ser parte del contenido del archivo.
 - En vez de marcar el fin del registro, se puede contar la cantidad de marcas de fin de campo. Si el registro tiene 4 campos, y se leyeron 4 marcas de fin de campo, significa que se terminó de leer el registro.
 - Otra alternativa es usar indicadores de longitud de campo/registro. Es decir, almacenar la longitud del campo/registro antes de su contenido para saber exactamente cuántos de los siguientes bytes corresponden al mismo.

Bajas

- Problema para recuperar el espacio físico:
 - Si se da de baja lógicamente un registro de 130 bytes, dejando ese espacio listo para ser reutilizado, surge un problema. No hay ninguna garantía de que el próximo registro que será dado de alta tenga una longitud de 130 bytes. Si el nuevo registro ocupa 10 bytes y se le asigna ese espacio, se desperdiciarán 120 bytes. En cambio, si el nuevo registro ocupase 140 bytes, no se le podría asignar ese espacio sin perder información.
- Técnicas para dar de alta un registro cuando hay espacio a reutilizar:
 - **Primer ajuste:** se le asigna el primer espacio disponible en donde quepa.
 - **Mejor ajuste:** se le asigna el espacio de menor tamaño en donde aún quepa.

- **Peor ajuste:** se busca el espacio de mayor tamaño, y se le asignan solo los bytes necesarios. Ejemplo: se da de alta un registro de 10 bytes, y el mayor espacio disponible a reutilizar es de 140 bytes. Se le asignarán los primeros 10 bytes al nuevo registro, y quedará un espacio de 130 bytes para las futuras altas.
- Comparación
 - Primer ajuste es la más rápida, ya que no requiere procesar el archivo en su totalidad.
 - Mejor ajuste resuelve rápidamente una vez que encuentra el espacio a asignar. Simplemente se asigna dicho lugar en su totalidad y se quita de la lista. En cambio, peor ajuste debe asignar solo el espacio requerido y actualizar el espacio restante.
 - Peor ajuste es la única que resulta en que los registros y campos solo se les asigne el espacio necesario.
- Fragmentación interna y externa
 - **Fragmentación interna:** se produce cuando a un elemento de dato se le asigna mayor espacio del necesario
 - La generan los registros de longitud fija. Ejemplo: si un campo nombre se define como `string[40]`, cualquier nombre con menos de 40 caracteres producirá fragmentación interna.
 - La genera también la técnica de mejor ajuste. Ejemplo: si se da de alta un registro de 130 bytes, y el espacio de menor tamaño donde cabe es de 131, se le asignará 1 byte de más al registro.
 - Dependiendo de la implementación, la puede generar la técnica de primer ajuste. Por convención, se dice que el primer ajuste asigna el espacio *completo*, por lo que generaría fragmentación interna.
 - **Fragmentación externa:** es el espacio disponible entre dos registros, pero que es demasiado pequeño para ser reutilizado. En otras palabras, es espacio no asignado y no usado.
 - La genera la técnica de peor ajuste. En el ejemplo de la misma, esos 130 bytes restantes se irán utilizando, hasta el punto en el que podrían quedar disponibles 11 bytes. Si se da de alta un registro de 10 bytes, y ese espacio es el único disponible, quedará 1 byte sin asignar, y es muy probable que ese byte nunca sea reasignado, ya que es muy pequeño.
 - Para ir recuperando esos bytes que “quedaron sueltos”, se ejecuta periódicamente un algoritmo que los recupera. Este tipo de algoritmo se denomina *garbage collector*.

Modificaciones

- ¿Qué pasa cuando se modifica un registro de longitud variable?
 - **Caso 1:** la longitud del campo modificado es igual a la original. No se genera ningún problema.
 - **Caso 2:** la longitud del campo modificado es menor a la original. Genera fragmentación interna, ya que se le está asignando al campo más espacio del que realmente ocupa. Podría generar fragmentación externa si se decide únicamente asignar el espacio realmente utilizado al registro, dejando el resto sin asignar.
 - **Caso 3:** la longitud del campo modificado supera a la original. No se puede utilizar el mismo espacio físico, se debe reubicar el registro.
 - Solución para simplificar los algoritmos:
 - La modificación se trata como un proceso de 2 partes:
 - Parte 1: se da de baja al registro viejo.
 - Parte 2: se da de alta el registro con el campo modificado, utilizando la política de recuperación de espacio elegida previamente.
-

Búsqueda de información

- Para buscar un dato, se consideran la cantidad de accesos a disco (almacenamiento secundario) necesarios para encontrarlo.
 - *Nota:* también se realiza una comparación, es decir, se revisa si el dato es el buscado. Este tiempo es despreciable porque la operación sucede en la memoria RAM.
- Técnicamente, la búsqueda de cualquier registro de longitud fija puede realizarse en un único acceso si se conoce su NRR.
 - **NRR:** es el *número relativo de registro*. Permite calcular la distancia en bytes desde el principio del archivo hasta cualquier registro.
 - **Nota:** es muy poco probable conocer el NRR del registro que se desea buscar de antemano, por lo que este caso de acceso directo se trata de una situación específica.

Búsqueda en distintos tipos de archivos

- **Archivo serie:** son aquellos en los que no existe ningún orden preestablecido de la información, exceptuando el orden físico.
 - Evaluación de la búsqueda lineal:

- **Mejor caso:** 1 acceso (se encuentra en la primera posición)
 - **Peor caso:** N accesos (no se encuentra el dato, o se encuentra en la última posición)
 - **Promedio:** $N/2$ accesos.
 - La *performance* es de orden N .
- Archivo ordenado:
 - Solo cambia si el criterio de búsqueda es igual al criterio de ordenación, y el dato no se encuentra en el archivo. En ese caso, la búsqueda terminará con la primera clave encontrada mayor a la buscada, y requerirá en promedio $N/2$ accesos.
 - La búsqueda sigue teniendo el mismo orden si se realiza de manera secuencial, porque el mejor y peor caso se mantienen iguales.
 - Si se está trabajando con registros de longitud fija, es óptimo utilizar la llamada búsqueda binaria.
 - Consiste en localizar el registro medio del archivo, y descartar la mitad del archivo que contiene claves mayores o menores según corresponda. Esto se repite hasta que el espacio de búsqueda se reduce a un registro.
 - Esta búsqueda no se puede usar con registros de longitud variable.
 - Evaluación de la búsqueda binaria:
 - **Mejor caso:** 1 acceso
 - **Peor caso:** $\log_2(N) + 1$ accesos
 - **Promedio:** $[(\log_2(N) + 1) / 2]$ accesos
 - La *performance* es de orden $\log_2(N)$. Se debe considerar también el costo de mantener el archivo ordenado.

¿Algoritmos de ordenación?

Índices

Índices primarios

- Definición
 - Un índice es una estructura de datos adicional que permite agilizar el acceso a la información almacenada en un archivo. En dicha estructura se almacenan las claves de los registros del archivo, junto al NRR asociado a la clave.

- Características de un índice primario.
 - Las claves que contiene son primarias, por lo que deben ser únicas
 - Todo índice debe estar ordenado.
 - Se almacena en memoria secundaria, es decir en otro archivo, distinto del archivo de datos.
 - Permite imponer un orden lógico sin que el archivo de datos esté físicamente ordenado.
 - Contiene registros de longitud fija.
- Acceso secuencial indizado a través de índice primario
 - Cuando se tiene un archivo con un índice primario, el acceso a un registro con clave primaria X se hace de la siguiente manera:
 - Se busca la clave X en el índice.
 - Como el índice está ordenado y tiene registros de longitud fija, en este paso se usa la [búsqueda binaria](#).
 - Se obtiene de allí el NRR del registro con clave X.
 - Se accede al registro con clave X en el archivo de datos.
- Ventajas
 - Como tiene menor tamaño que el archivo de datos, y siempre tiene registros de longitud fija, mejora considerablemente la *performance* de búsqueda.
 - Siempre se pueden realizar búsquedas binarias.

Operaciones

- **Creación:** el índice se crea junto con el archivo de datos, ambos vacíos y solo con el registro encabezado.
- **Altas**
 - Cuando se da de alta un nuevo registro, se agrega al final del archivo de datos, porque este no tiene que estar ordenado.
 - Luego, el par (NRR + clave primaria) se inserta de manera ordenada al índice.
 - Si el archivo de datos fuese de registros de longitud variable, se almacenaría la dirección del primer byte en vez del NRR.
- **Modificaciones**
 - **La clave primaria nunca se modifica.** Es por esto que, si el archivo es de registros de longitud fija, el índice no cambia ante una modificación del archivo de datos.
 - Si el archivo es de registros de longitud variable, pero la modificación no cambia la longitud del registro, el índice no cambia.

- Si el archivo es de registros de longitud variable, y la modificación cambió su tamaño, el registro cambiará de lugar en el archivo de datos. Por lo tanto, la dirección del primer byte tendrá que cambiar en el índice.
- **Bajas**
 - Al borrar un registro del archivo de datos, se borra también del índice primario, ya sea lógica o físicamente.
 - No se usan las políticas de recuperación de espacio en el índice, ya que este debe estar ordenado en todo momento.

Índices para claves candidatas

- Las claves candidatas son aquellos campos de un registro que no admiten repeticiones (como la clave primaria), pero que no fueron seleccionados como clave primaria.
- Los índices son similares a los primarios.

Índices secundarios

- Utiliza claves secundarias, es decir, claves que permiten valores repetidos.
- Es una estructura adicional que permite relacionar una clave secundaria con una o más claves primarias.
- La razón por la que a través del índice secundario se accede a la clave primaria, y no a la dirección del registro, es la siguiente:
 - Si el índice secundario permitiera acceder a la dirección del registro, tendría que modificarse cada vez que cambie de posición dicho registro en el archivo de datos.
 - Esto es relevante porque se puede tener más de un índice secundario, y actualizarlos todos cada vez que cambia de lugar un registro resulta costoso.

Operaciones

- **Creación:** al igual que los primarios, se crean vacíos junto al archivo de datos. Contienen solo el registro encabezado.
- **Altas:** siempre que se realiza un alta en el archivo de datos de una nueva clave secundaria, se realiza una inserción ordenada en el índice secundario.
 - Esta operación es costosa en términos de *performance* si el índice está en almacenamiento secundario. Particularmente porque es posible que el índice secundario tenga dos criterios de ordenación.

- **Modificaciones:** si se modifica la clave secundaria, se debe reordenar el índice secundario. Si se cambia cualquier otro campo del registro (excepto la clave primaria), no se genera ningún cambio en el índice.
- **Bajas:** existen dos maneras de modificar el índice secundario luego de una baja en el archivo de datos, y la correspondiente eliminación de dicha clave primaria en el índice primario.
 - **Alternativa 1:** se realiza una baja lógica o física en el índice secundario, similar a lo que sucede en el primario.
 - **Alternativa 2:** no se modifica en absoluto el índice secundario. Simplemente se borra la referencia en el índice primario.
 - **Ventaja:** no se tienen que modificar los índices secundarios luego de una baja.
 - **Desventajas:**
 - Los datos borrados continúan ocupando espacio. Esto es un problema si el archivo de datos es altamente volátil. Es decir, si sufre de bajas de manera frecuente.
 - Puede suceder que el usuario busque en el índice secundario un dato ya borrado. En ese caso, la búsqueda será exitosa en este primer índice, y recién se informará del error al ir a buscar el dato al índice primario (en donde sí se borró).

Alternativas

- Alternativa 1
 -

Clave secundaria (marca)	Clave primaria (patente)
FIAT	AA 954 FG
FIAT	AB 875 SD
HONDA	AA 457 LB
PEUGEOT	AA 474 CD
PEUGEOT	AC 645 LP

- Cuando la clave secundaria se repite, hay un segundo criterio de ordenación, establecido por la clave primaria.
 - Por eso todos los FIAT están ordenados por patente, y lo mismo con los Peugeot
- **Problemas:**

- El índice se debe reacomodar con cada registro ingresado, aún si la clave secundaria del mismo ya se encuentra almacenada.
- Al almacenar distintas ocurrencias de la misma clave secundaria, se desperdicia espacio. Esto reduce la posibilidad de que el índice quepa en memoria principal.
- La búsqueda binaria requiere un costo extra, ya que la clave secundaria tiene repeticiones. Si el usuario está buscando un FIAT, y lo hace usando búsqueda binaria, el algoritmo puede encontrar una aparición de un FIAT, pero no será necesariamente la primera ni la última.

- Alternativa 2

-

Clave secundaria	Vector de claves primarias
FIAT	AA 954 FG, AB 875 SD
HONDA	AA 457 LB, [_____]
PEUGEOT	AA 474 CD, AC 645 LP

- **Problema:** la elección del tamaño del vector se debe hacer de antemano. Esto se debe a que las claves primarias deben ser de longitud fija para poder realizar búsqueda binaria.
 - Esto puede generar fragmentación interna. En el ejemplo de la tabla de arriba, si el tamaño del vector es 2, el vector de la marca Honda tiene un espacio sin utilizar.
 - También puede suceder que el espacio del vector resulte insuficiente.

- Alternativa 3

- Se tienen 2 archivos además del archivo de datos.
 - Uno contiene las claves secundarias sin repeticiones y el puntero hacia el primer elemento de la lista invertida contenida en el otro archivo. Dicha lista invertida contiene todas las apariciones de la clave secundaria mediante la cual se accedió, con sus respectivas claves primarias. Es una versión dinámica de la alternativa 2.
 - El otro archivo contiene todas las claves primarias y un enlace al siguiente. El “siguiente” vendría a ser la clave primaria de la próxima aparición de la clave secundaria procesada.
- Archivo 1 (índice secundario)

Clave secundaria	Puntero
------------------	---------

FIAT	0
HONDA	2
PEUGEOT	3

- Archivo 2 (listas invertidas)

NRR	Clave primaria	Enlace al siguiente
0	AA 954 FG	4 (siguiente fiat)
1	AC 645 LP	-1 (no hay más peugeot)
2	AA 457 LB	-1 (no hay más honda)
3	AA 474 CD	1 (siguiente peugeot)
4	AB 875 SD	-1 (no hay más fiat)

- Ventajas
 - El único reacomodamiento en el índice se produce cuando se agrega una nueva clave secundaria.
 - Si se agregan o borran datos de una clave secundaria ya existente, solo se modifica el archivo 2. En cada operación solo se modificaría una de las listas.
 - Como son dos archivos, uno podría almacenarse en memoria secundaria, liberando la RAM. Particularmente, el archivo 2 (listas invertidas) suele sufrir más cambios, por lo que es conveniente que esté en memoria principal
- Desventaja
 - Si existiesen muchos índices secundarios, mantener 2 archivos por cada uno podría resultar costoso.

Índices selectivos

- Es posible tener un índice que incluya únicamente aquellas claves secundarias asociadas a la información con mayor interés de acceso. Por ejemplo, se podría tener un índice que solo contenga los autos con clave secundaria "FIAT".
-

Árboles binarios

- Es una estructura de datos dinámica no lineal, en la cual cada nodo puede tener a lo sumo 2 hijos.
- La estructura de árbol resulta útil como método de almacenar índices. Existen distintos tipos de árboles con sus respectivas ventajas y desventajas.
- Tiene sentido si los datos están ordenados (menores a la izquierda y mayores a la derecha)
- Es posible almacenar un árbol binario en un archivo. Simplemente se hace un “file of record”, en el que el *record* contiene el dato, y el NRR (*integer*) de sus 2 hijos.
- *Performance*
 - Idealmente, la búsqueda de un elemento en un árbol binario es de orden $\log_2(N)$, ya que por cada revisión se descarta la mitad del archivo restante. Esta *performance* es comparable con la búsqueda binaria sobre un archivo ordenado.
 - A diferencia de un índice ordenado almacenado sin un árbol, la operación de alta no requiere la reorganización de todo el archivo. Simplemente se agrega el nuevo elemento en una hoja, y se actualiza el padre para que tenga la dirección del nuevo nodo.
 - Requiere únicamente de $\log_2(N)$ lecturas para encontrar al padre, y 2 operaciones de escritura (el nuevo nodo y el NRR del padre).
 - Lo mismo sucede con las bajas. Toda baja se realiza sobre un nodo terminal. Si el elemento a borrar está en un nodo interno, se intercambia con el menor de sus hijos mayores. Luego se elimina el dato del nodo terminal, que no tiene hijo izquierdo.
 - Requiere de $\log_2(N)$ lecturas y 2 escrituras (intercambio de datos + marca de borrado sobre el elemento correspondiente).

Problema de árboles binarios

- Se desbalancean fácilmente.
 - Se considera balanceado a un árbol que tiene todas sus hojas en el mismo nivel.
 - Imaginemos que en un árbol binario, se realizan las siguientes inserciones: 100, 99, 98, 97, 96, etc hasta 1
 - Como cada elemento es menor al anterior, siempre se insertarán como hijos izquierdos. Esto llevará a que la búsqueda sobre el árbol ya no sea de orden logarítmico, sino

que será de orden lineal. El árbol se degenera en una lista simplemente enlazada

- El problema con este ejemplo es que la raíz (100) era el mayor de todos los elementos. Lo ideal sería saber cuál será la mejor raíz antes de insertar los elementos. Esto no es posible con un simple árbol binario.
- Aún en el mejor de los casos, en el que el árbol está balanceado, $\log_2(N)$ accesos para llegar a las hojas resulta un número demasiado alto de accesos a disco para una búsqueda.

Árboles AVL

- Un árbol balanceado en altura es un árbol binario, que sigue una regla específica: la diferencia entre el camino más corto y más largo de entre la raíz y una hoja no puede diferir en más que un determinado número.
- Para un árbol AVL, dicho determinado número es 1. Es decir, podría suceder que el camino más corto entre la raíz y una hoja sea 5, y que el más largo sea 6. Sin embargo, si se realiza una inserción que crea un nuevo nivel, el camino más largo pasaría a ser 7. Como ahora ambos caminos difieren en su longitud por 2, se debe *rebalancear* el árbol, para que vuelva a cumplir la propiedad establecida.
 - Estos algoritmos de rebalanceo tienen un costo de accesos a disco elevado, por lo que los árboles AVL no resultan una buena opción para almacenar los índices.
- La *performance* de la búsqueda en el peor caso es de $1.44 \cdot \log_2(N+2)$.
 - Es ligeramente peor que la de un árbol binario balanceado, pero se mantiene con menor costo.
 - Aunque el costo de mantenerlo sea menor, sigue siendo alto, por lo que no resulta óptimo para almacenar los índices.
- Como siguen siendo binarios, el acceso a las hojas sigue siendo de $\log_2(N)$, lo que es inaceptablemente alto.

Paginación de árboles binarios

- El concepto de [*buffering*](#) resulta útil para traer datos desde un árbol almacenado en memoria secundaria a la RAM.
- Árboles binarios paginados
 - El árbol se divide en páginas, cada una del mismo tamaño que el buffer.

- Cada página contiene un conjunto de nodos ubicados en direcciones físicamente cercanas entre sí.
- De esta manera, cada vez que se accede al disco, se transmite una página completa a la RAM.
- Por lo tanto, cuando se accede a un nodo, y se transmite su página completa (por ejemplo de 7 nodos), se logran transmitir 7 nodos con un solo acceso a disco.
 - ¿Por qué no se consideran como 7 accesos? Porque, aunque sí se acceden a 7 direcciones físicas diferentes, éstas están físicamente cerca entre ellas. Esto significa que los desplazamientos realizados para su acceso son despreciables.
- La *performance* final de la búsqueda en un árbol binario paginado es de $\log_{k+1}(N)$, con k siendo la cantidad de nodos que hay en cada página.

Problema de los árboles paginados

- Para que el uso de páginas tenga sentido, los nodos que contiene cada una deben estar físicamente cerca entre ellos.
 - La única manera de que esto suceda es que (tomando un ejemplo de 7 nodos por página), las 7 claves lleguen consecutivamente. Como esto es imposible de controlar, no resulta una buena opción.
 - En cambio, lo que se hace es que, una vez que se inserta un nodo en una nueva página, se reservan los 6 nodos restantes en las direcciones cercanas en el disco. Esto genera [fragmentación interna](#), y resulta en que cada nodo se almacene en la página que le corresponde, en vez de la primera disponible.
 - Los algoritmos para mantener este tipo de árbol implican un alto costo de implementación y de *performance*.
-

Árboles multcamino

- Un árbol multcamino de orden M es una estructura de datos en la cual cada nodo puede contener $M - 1$ elementos y M descendientes.
- Hacer que cada nodo contenga más de un elemento es otra manera de resolver el concepto de páginas. Es decir, es posible hacer que, en vez de tener 7 nodos de 1 clave c/u por página, existan nodos con 7 claves cada uno.

- Estos árboles aún requieren ser balanceados. Así nacen los árboles B, B+ y B*, que son árboles multicamino balanceados.

Árboles B

- Son árboles multicamino con una construcción especial (de abajo hacia arriba) que se pueden mantener balanceados a bajo costo.
- Se considera que todas las operaciones de un árbol B (consulta, alta, baja o modificación) se realizan en términos aceptables de *performance*.
- Propiedades de un árbol B de orden M
 - Cada nodo del árbol puede contener, como máximo, M descendientes y $M - 1$ elementos.
 - La raíz o no tiene hijos o tiene por lo menos 2.
 - Un nodo con x hijos contiene $x - 1$ elementos.
 - Los nodos terminales (hojas) tienen, entre $\lfloor M/2 \rfloor - 1$ elementos y $M - 1$ elementos.
 - Los nodos que no son terminales ni raíz tienen, como mínimo, $\lfloor M/2 \rfloor$ hijos
 - Todos los nodos terminales se encuentran al mismo nivel.

Implementación en Pascal

const

orden = 255;

type

reg_arbol_b = *record*

hijos: *array* [0..*orden*] of *integer*;

claves: *array* [1..*orden*] of *tipo_de_dato*;

nro_registros: *integer*;

end;

Eficiencia de búsqueda

- La eficiencia de búsqueda de un árbol B se mide en la cantidad de accesos a disco necesarios para localizar un elemento, o para determinar que dicho elemento no se encuentra.
 - **Mejor caso:** 1 lectura (se encuentra en la raíz)
 - **Peor caso:** H lecturas (se encuentra en una hoja o no se encuentra)
- Teniendo en cuenta que un acceso a disco implica leer un nodo del árbol, la cantidad de accesos siempre estará en el rango entero $[1; H]$, con H siendo la altura del árbol

- Para determinar la eficiencia en términos de N , resulta necesario acotar el valor de H . Es decir, determinar el valor máximo de H para una cantidad N de registros.
- El resultado de una demostración matemática (pág 123-124 del libro) para la obtención de la cota es la siguiente inecuación:
 - $H < \log_{\lfloor M/2 \rfloor} [(N + 1)/2]$
 - Para los valores $N = 1.000.000$ y $M = 512$, obtenemos que la máxima altura del árbol, y por lo tanto la cantidad máxima de accesos a disco a la hora de realizar una búsqueda, es de 3,37. Esto es una mejora considerable respecto a métodos anteriores.

Eficiencia de inserción

- **Mejor caso (no hay *overflow*)**
 - H lecturas (siempre se inserta en nodo terminal).
 - 1 escritura.
- **Peor caso (*overflow* se propaga hasta la raíz, aumentando la altura del árbol)**
 - H lecturas.
 - $(2 * H) + 1$ escrituras.
- **Estudios**
 - Según estudios realizados, con árboles de orden 10, la cantidad de *overflows* es de 25%. En cambio, si el orden es 100, la cantidad de *overflows* se reduce a 2%. Es decir, 98 de cada 100 inserciones tratan con el mejor caso, y el restante 2% tratan con algún tipo de *overflow* (no necesariamente propagándose hasta la raíz).

Eficiencia de eliminación

- **Mejor caso (no hay *underflow*)**
 - H lecturas.
 - 1 escritura.
- **Peor caso (*underflow* se propaga hasta la raíz, reduciendo la altura del árbol)**
 - $(2 * H) - 1$ lecturas (por nivel, se lee al nodo y a su adyacente para realizar la concatenación/fusión. Excepto por la raíz que no tiene adyacentes).
 - $H - 1$ escrituras (1 por nivel, pero la cantidad de niveles se reduce a $H - 1$).

Eficiencia de modificación

- La modificación se trata como una baja, seguido de una inserción con las modificaciones deseadas. De esta manera, la *performance* será el resultado de sumar una operación de baja seguida de una inserción.

Árboles B*

- Un árbol B* es un árbol balanceado con las siguientes propiedades
 - Cada nodo del árbol puede contener, como máximo, M hijos y $M - 1$ elementos.
 - La raíz o no tiene hijos o tiene por lo menos 2.
 - Un nodo con x hijos contiene $x - 1$ elementos.
 - Los nodos terminales tienen entre $\lceil (2M - 1)/3 \rceil - 1$ elementos y $M - 1$ elementos.
 - Los nodos que no son terminales ni raíz tienen, como mínimo, $\lceil (2M - 1)/3 \rceil$ hijos.
 - Todos los nodos terminales están en el mismo nivel.
- Tratamiento de *overflow*
 - A diferencia de los árboles B, en los que siempre se genera un nuevo nodo al producirse *overflow*, en los B* se intenta redistribuir antes.
 - Cuando un nodo se completa, reubica sus elementos usando un nodo adyacente hermano.
 - Si el adyacente hermano también está lleno, se dividen esos 2 nodos completos en 3 nodos, que tendrán $\frac{2}{3}$ de su capacidad llena.
 - Excepción
 - Cuando el árbol tiene un solo nivel, y se completa la raíz, no es posible redistribuir. Entonces, el nodo raíz genera 2 hijos, que están llenos a la mitad únicamente. Este es el único caso en el que no se respeta la cantidad mínima de elementos de $\lceil (2M - 1)/3 \rceil$.
- Las operaciones, con excepción de las altas, son similares a los árboles B.

Políticas de *overflow*

- Estas políticas determinan con qué nodos hermanos adyacentes se intentará redistribuir en caso de *overflow*.
- Política de un solo lado
 - Para el ejemplo, vamos a suponer que se eligió la política derecha.
 - En caso de *overflow*:

- El nodo completo intenta redistribuir con solo su hermano adyacente derecho. Si ese hermano también está lleno, se debe realizar el split a 3 nodos llenos a $\frac{2}{3}$ de capacidad. Nunca se intenta redistribuir con el hermano izquierdo, excepto que sea su único hermano.
- Política de un lado u otro
 - Para el ejemplo, tomemos la política de derecha o izquierda.
 - En caso de *overflow*:
 - El nodo completo intentará redistribuir con su hermano adyacente derecho. Si dicho hermano está lleno, se intentará redistribuir con el adyacente izquierdo. Si ambos adyacentes están llenos, se dividirán el nodo y su hermano derecho en 3 nodos llenos a $\frac{2}{3}$ de su capacidad.
- Política de un lado y el otro
 - Para el ejemplo, tomamos la política de derecha e izquierda.
 - En caso de *overflow*:
 - El nodo completo intentará redistribuir con su hermano adyacente derecho. Si dicho hermano está lleno, se intentará redistribuir con el adyacente izquierdo. Si ambos están llenos (y acá está la diferencia con la política anterior), se toman los 3 nodos y se generan 4 nodos con $\frac{3}{4}$ partes completas cada uno.
 - Esta política completa más los nodos del árbol, y hace que el mismo tenga menor altura. Sin embargo, también necesita un mayor número de operaciones de E/S sobre disco para ser implementada.

Performance de inserción en B*

- No se considera la lectura ni escritura del nodo padre, porque es igual en todos los casos. Siempre se lee para llegar a las hojas, y siempre se escribe al redistribuir o dividir.

	Mejor caso	Peor caso
Un lado	RRWW	RRWWW
Un lado u otro	RRWW	RRRWWW
Un lado y el otro	RRWW	RRRWWW

Árboles B+

- Los árboles B permiten el acceso a un elemento con una *performance* aceptable. Sin embargo, si se quiere obtener una lista ordenada de todos los elementos, se tendría que recorrer el árbol de manera in-order. Esto resulta en una *performance* inaceptable.
- La solución son los árboles B+:
 - En estos, los datos almacenados en los nodos internos actúan únicamente como separadores. Este es el **conjunto índice**.
 - Todas las claves de los registros realmente almacenados en el archivo viven en las hojas del árbol B+. Dichas hojas están enlazadas, de manera que desde cualquier hoja, se pueden obtener las claves de todas las hojas siguientes. Como las claves dentro de cada hoja están ordenadas, esto permite la recuperación ordenada de los datos. El conjunto real de datos, almacenado en las hojas, se denomina **conjunto secuencia**.
 - Al realizar la eliminación de una clave, esta solo se efectúa si dicha clave se encuentra en el conjunto real de datos, es decir, en las hojas del árbol.

Árboles B+ de prefijos simples

- Son árboles B+ en los que los separadores están representados por la mínima expresión posible de la clave, que permita decidir si la búsqueda se realiza hacia la izquierda o hacia la derecha. Esto se hace con el fin de ahorrar espacio físico, y aumentar la cantidad de separadores que caben en un nodo.
- Se usan únicamente para índices que contienen claves alfanuméricas (*strings*).
- En las hojas, las claves reales se almacenan en su totalidad.

Relación entre árboles y buffers

- La capacidad que tienen los nodos de un árbol está determinada por la cantidad de información que puede manejar el sistema operativo, a través del uso de *buffers*.
- Los buffers que se mantienen en la memoria principal los determina una política. Una de ellas es la política LRU, que elimina de la memoria principal a aquellos buffers que menos se usaron recientemente. De esta manera, si se está tratando con un árbol, es probable que la raíz sea el nodo más recientemente usado, y se encuentre en la memoria principal. Cuando esto

sucede, el número de accesos a disco se reduce en 1, ya que no se accede al disco para obtener la raíz.

Dispersión (*hashing*)

- Es una técnica que convierte a una clave en un número pseudo-aleatorio. Luego, a través de la *función de hash*, convierte ese número en una dirección base única en donde se almacenará la misma.
- Asegura, en promedio, un único acceso para recuperar la información de la clave buscada.

Propiedades

- No requiere almacenamiento adicional. Esto significa que cuando se elige organizar el archivo mediante dispersión, es el archivo de datos el que resulta disperso. La dispersión se genera a partir de la clave primaria, por lo que no hace falta una estructura auxiliar para acceder rápidamente a la información.
- Facilita la inserción y eliminación rápida de registros en el archivo. En general, solo se requiere de 1 acceso a disco para realizar dichas operaciones.
- La búsqueda también se reduce, en promedio, a un solo acceso. No es posible asegurar que este sea siempre el caso.

Limitaciones

- No se puede aplicar a archivos con registros de longitud variable. Cuando se obtiene la dirección de almacenamiento a partir de la función de *hash*, asociada a esta hay un espacio de almacenamiento conocido y acotado. No puede suceder que el registro a almacenar no quepa en dicho espacio.
- No es posible obtener un orden lógico de los datos. Es decir, obtener un listado ordenado de todos los elementos de un archivo, usando la función de hash para acceder a los mismos, sería inaceptable en términos de *performance*.
- No es posible tratar con claves secundarias. Esto se debe a que 2 registros con la misma clave secundaria, una vez que se les aplica la función de hash, resultan en la misma dirección de memoria.
 - Tampoco es posible tratar con [claves candidatas](#). Esto se debe a que, un registro contiene tanto la clave primaria como la clave candidata en

sus campos. Si se usase *hashing* para las 2, la función devolvería distintas direcciones de memoria secundaria para un mismo registro.

Tipos de dispersión

- **Hashing con espacio de direccionamiento estático**
 - Es una política en donde el espacio disponible para dispersar los registros de un archivo de datos está fijado previamente. Por lo tanto, se conoce de antemano la cantidad de registros que caben en el archivo.
 - La función de *hash* devuelve una dirección física posible dentro del espacio fijado.
- **Hashing con espacio de direccionamiento dinámico**
 - Es una política donde el espacio disponible para dispersar los registros de un archivo de datos aumenta o disminuye en función de las necesidades de espacio que en cada momento tiene el archivo.
 - La función de *hash* devuelve un valor intermedio, que será utilizado para obtener una dirección física posible para el registro dentro del archivo. Estas direcciones no están establecidas de antemano.

Parámetros de la dispersión

- El método de dispersión con espacio de direccionamiento estático, tiene 4 parámetros que definen su comportamiento. Estos son: función de *hash*, tamaño de cada nodo de almacenamiento, densidad de empaquetamiento y método de tratamiento de *overflow*.

Función de *hash*

- Es una función que transforma un número, que representa una clave primaria, en otro valor dentro de un determinado rango, que se usa como dirección física para almacenar el registro.
- Idealmente, devolvería direcciones de manera uniforme.
- Se debe asegurar que la dirección retornada por la función esté dentro del rango determinado, mapeando el resultado al rango de direcciones disponibles.
- Claves sinónimas entre sí: son 2 claves que, una vez que se les aplica la función de *hash*, retornan la misma dirección de memoria secundaria. A esto se le llama colisión.
- Se busca que la función de *hash* funcione de tal manera que exista la menor cantidad de claves sinónimas posibles. Para lograr esto, se podría:

- Desarrollar un algoritmo de dispersión perfecto, que no genere colisiones. Esto es extremadamente difícil, por lo que no se considera una opción válida.
- Minimizar el número de colisiones problemáticas (*overflow*) a una cantidad aceptable, y tratar a las mismas como casos especiales. Los métodos para hacerlo son los siguientes:
 - Distribuir los registros de la forma más aleatoria posible.
 - Utilizar más espacio en disco. Por ejemplo, en vez de reservar 10 espacios para los 10 registros que se desea almacenar, podrían reservarse 100 direcciones de memoria para los mismos. Esto genera desperdicio de espacio, pero minimiza las colisiones.
 - Almacenar más de un registro por cada dirección física del archivo. En este caso, una colisión no necesariamente generaría *overflow*, ya que 2 claves sinónimas podrían almacenarse en la misma dirección física. La capacidad de cada dirección debe estar fijada de antemano.

Tamaño de cada nodo/celda/*bucket* de almacenamiento

- La cantidad ideal de registros que caben en la celda cuya dirección es devuelta por la función de hash, determina la posibilidad de transferencia de información de entrada/salida entre el disco y la RAM. Esto es similar al [criterio para determinar el tamaño de un nodo de un árbol B](#).
- Mientras más registros quepan en una celda:
 - - *overflow*
 - + fragmentación
 - + lenta la búsqueda dentro de la celda (despreciable porque sucede en RAM)

Densidad de empaquetamiento

- Es la relación entre el espacio disponible para el archivo de datos y la cantidad de registros que realmente integran dicho archivo. Matemáticamente:

$$○ DE = \frac{r}{RPN * n}$$

- **r**: cantidad de registros que integran el archivo.
- **RPN**: cantidad de registros que caben en un nodo (registros por nodo).
- **n**: cantidad de nodos en el archivo.

- Mientras la DE sea menor:
 - - *overflow*
 - + desperdicio de espacio

Método de tratamiento de *overflow*

- El *overflow* ocurre cuando un registro es direccionado a una celda que no dispone de capacidad para almacenarlo. Para solucionar esto, se debe:
 - Encontrar otro lugar para almacenarlo.
 - Asegurarse que, cuando se busque dicho registro, se encuentre en el lugar en donde efectivamente se almacenó, y no donde “debería ir” (su celda nativa).

Estudios de *overflow*

- Se han realizado estudios cuyos resultados dictan que, si en cada nodo caben 100 registros, y la DE es menor a 75%, los casos de *overflow* son casi inexistentes.

DE	1	2	5	10	100
10%	4.8	0.6	0.0	0.0	0.0
20%	9.4	2.2	0.1	0.0	0.0
30%	13.6	4.5	0.4	0.0	0.0
40%	17.6	7.3	1.1	0.1	0.0
50%	21.4	10.4	2.5	0.4	0.0
60%	24.8	13.7	4.5	1.3	0.0
70%	28.1	17.0	7.1	2.9	0.0
75%	29.6	18.7	8.6	4.0	0.0
80%	31.2	20.4	10.3	5.3	0.1
90%	34.1	23.8	13.8	8.9	0.8
100%	36.8	27.1	17.6	12.5	4.0

Posibilidades para resolver el *overflow*

Saturación progresiva

- Una vez que se produce *overflow* en un *bucket*, se avanza en el archivo hasta encontrar el nodo más cercano que está libre. Se asigna ese nodo a la clave que produjo *overflow*.
- Si se busca a la clave que produjo *overflow*, se hace lo siguiente:
 - Se busca en su *bucket* nativo
 - Si el *bucket* está lleno, pero no contiene la clave, se continúa leyendo todos los *buckets* hasta encontrarla. La búsqueda se detiene al encontrarla, o al leer un *bucket* que no esté completo.
 - Para asegurarse de que toda clave siempre se pueda encontrar, sin importar las bajas que se hayan realizado, una vez que un *bucket* se llena, se debe almacenar en él una marca de que estuvo completo.
- **Problemas:**
 - Genera áreas de *overflow*. Los *buckets* "al final" del archivo reciben más claves, porque, además de recibir sus claves nativas, pueden recibir claves nativas a cualquier otro bloque en el que se haya producido *overflow*. Esto hace que la distribución de claves no sea uniforme.
 - En un caso extremo, la búsqueda de una clave puede requerir de N accesos.

Saturación progresiva encadenada

- Es similar al primer método, con una única diferencia. Una vez que la clave que produjo *overflow* se almacenó en el *bucket* más cercano, en el *bucket* "nativo" se almacena un enlace al comienzo de una lista encadenada de claves sinónimas, que se almacenaron en direcciones distintas a la dirección base. De esta manera, se puede acceder a las claves que produjeron *overflow* sin tener que pasar por todos los *buckets* del medio. Es ligeramente mejor en cuanto a *performance* que la saturación progresiva, pero sigue teniendo el mismo problema, ya que también genera áreas de *overflow*.

Dispersión doble

- Se definen 2 funciones de hash completamente independientes: $F_1(x)$ y $F_2(x)$

- **Nota:** que sean independientes significa que si $F_1(x)$ de las claves XXX, YYY dieron como resultado 50, y $F_2(x)$ de XXX dio como resultado 20, esto **NO** significa que $F_2(x)$ de YYY también vaya a dar 20.
- Inicialmente se usa únicamente $F_1(x)$. Una vez que al insertar una clave, se produce *overflow*, se aplica $F_2(x)$ a la clave. Esta nueva función va a devolver un número, que se va a usar como desplazamiento al resultado que había devuelto $F_1(x)$. El desplazamiento se sigue sumando hasta encontrar un *bucket* libre.
- **Ventaja:** no genera áreas de *overflow*.
- **Desventaja:** Las claves quedan alejadas físicamente de sus direcciones nativas. Esto rompe la lógica del principio de localidad espacial. Por lo tanto, aumenta el desplazamiento de la cabeza lectora/grabadora del disco, aumentando la latencia entre pistas, y el tiempo de respuesta.

Área de desborde separada

- Se define el mayor número primo que sea menor a la cantidad de *buckets*.
- **Ejemplo:** 2500 *buckets*, el mayor primo menor a 2500 es 2477.
 - La función de hash va a devolver siempre números entre 0 y 2476.
 - Los 23 *buckets* que sobran se quedan en un área separada, reservada para casos de *overflow*.
 - Cuando se inserta una clave, y su *bucket* nativo está lleno, se almacena en la primera dirección libre dentro de los *buckets* de *overflow* [2477..2499]. También, en el *bucket* nativo, se almacena un enlace al comienzo de la lista de claves sinónimas, almacenadas en el área de desborde.
- **Ventaja:** Todas las claves almacenadas entre el 0 y el 2476 están en su *bucket* nativo, lo que significa que siempre puedo acceder a ellas en 1 solo acceso.

Hashing con espacio de direccionamiento dinámico

- Aunque se ha determinado que, con una DE menor a 75%, los casos de *overflow* son poco comunes, ¿qué pasa cuando se comienza a superar el umbral de 75%?
 - El *overflow* se hará cada vez más común. Una potencial solución a esto es el *hashing* con espacio de direccionamiento dinámico, en el que el espacio del archivo no se determina de antemano. Por lo tanto,

no existe la DE. A continuación se explica el método de *hash* extensible.

- En este método, la función de *hash* no devuelve una dirección, sino que devuelve una secuencia de bits. La longitud de la misma está predeterminada, y limitada por el espacio máximo que puede llegar a ocupar el archivo.
 - Por ejemplo, si la función devuelve 32 bits, se podrán almacenar, como máximo, 2^{32} nodos. Sin embargo, este espacio no se reserva en su totalidad desde el principio, sino que se reserva a medida que se almacenan registros.
 - Utiliza una estructura auxiliar que es almacenada en memoria, rompiendo una de las [propiedades del hashing](#).
 - Es una tabla que contiene la dirección física de cada nodo, a las que se puede acceder usando la secuencia de bits asociada a la clave.
 - Cabe destacar que la secuencia de bits no se usa en su totalidad, sino que se usan únicamente los bits necesarios, lo que es determinado por la cantidad de nodos almacenados. Es decir, si solo hay 2 nodos almacenados, sería innecesario identificar cada uno con una secuencia de 32 bits, ya que con un solo bit alcanzaría. En este caso, se usaría el bit menos significativo de la secuencia de 32 bits.
 - Ventajas
 - Siempre se accede al nodo deseado en un único acceso.
 - Al ser dinámico, el archivo nunca se tiene que dispersar de nuevo, lo que es altamente costoso.
 - Desventajas
 - La cantidad de entradas de la tabla es de 2^i , con i siendo el número de bits actuales de la tabla. Esto significa que crece exponencialmente, lo que puede ser un problema, al traer la tabla a la RAM.
 - La tabla auxiliar se almacena en el disco, pero por cuestiones de *performance*, se trae a RAM cuando se quiere trabajar con ella. La RAM es un almacenamiento limitado y volátil. La desventaja de que sea volátil es que es vulnerable a que, por ejemplo, se corte la luz mientras se trabaja con la tabla, y se pierda el progreso.
-

Conclusiones

Organización	Acceso a un registro por clave primaria	Acceso a todos los registros por clave primaria de manera ordenada
Ninguna	Muy ineficiente	Muy ineficiente
Secuencial	Poco eficiente	Eficiente
Secuencial indizada	Muy eficiente	El más eficiente
<i>Hash</i>	El más eficiente	Muy ineficiente